How to adopt the Bill of Behaviour into your daily workflow:

for transparent security

AMBASSADOR

CLOUD NATIVE COMPUTING FOUNDATION

Key Researcher

SBA Research

Special Thanks to:

Redpill Linpro Cloud

Dr Constanze Roedig

This project is partially funded by EOSC Future INFRAEOSC-03-2020 Grant 101017536, part of ASC and TUW

TU WIEN TECHNISCHE UNIVERSITÄT WIEN

ASC Austrian Scientific Computing

EUROPEAN OPEN SCIENCE CLOUD

GÉANT

Federal Ministry Innovation, Mobility and Infrastructure Republic of Austria

Federal Ministry Economy, Energy and Tourism Republic of Austria

FFG Promoting Innovation.

vienna business agency

For the City of Vienna

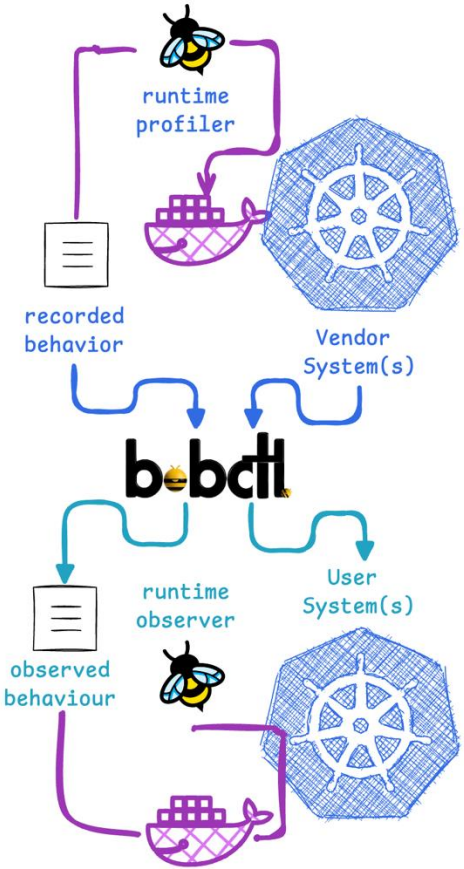
netidee OPEN INNOVATIONS

FWF Austrian Science Fund

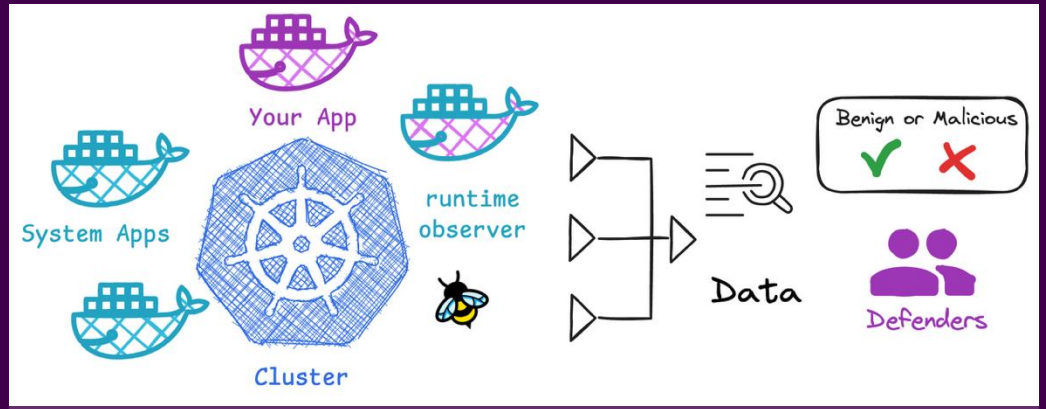
European Commission

k8sstormcenter

(S)BoB as a new standard - SBOM for runtime



- Software Bill of Behavior
  - How do you detect tampering?
  - Who writes runtime rules?
  - Who should write runtime rules?
  - Can we transfer runtimes rules from A -> B?
  - Alerting vs Enforcing
  - Attack Surface Shrinking
  - Integration into the ecosystem
  - What you can do today to benefit & help!

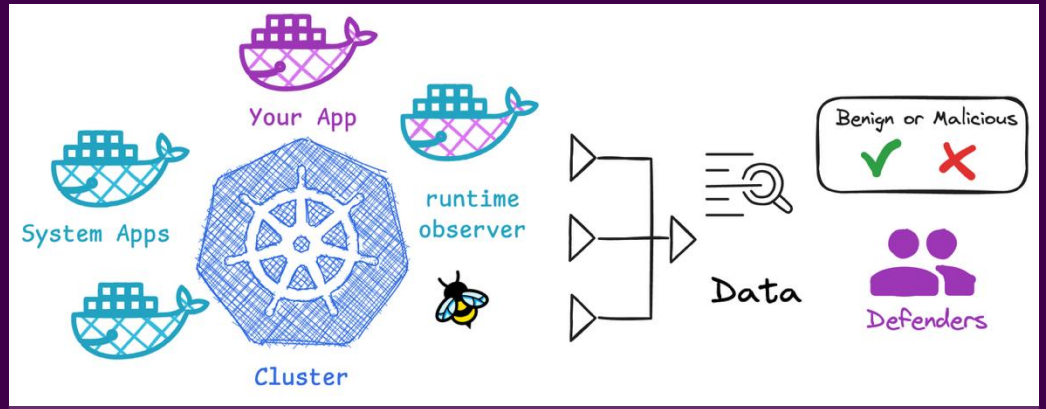


How do you detect tampering?

observe: get some data out

analyze: was this stuff bad?

k8sstormcenter



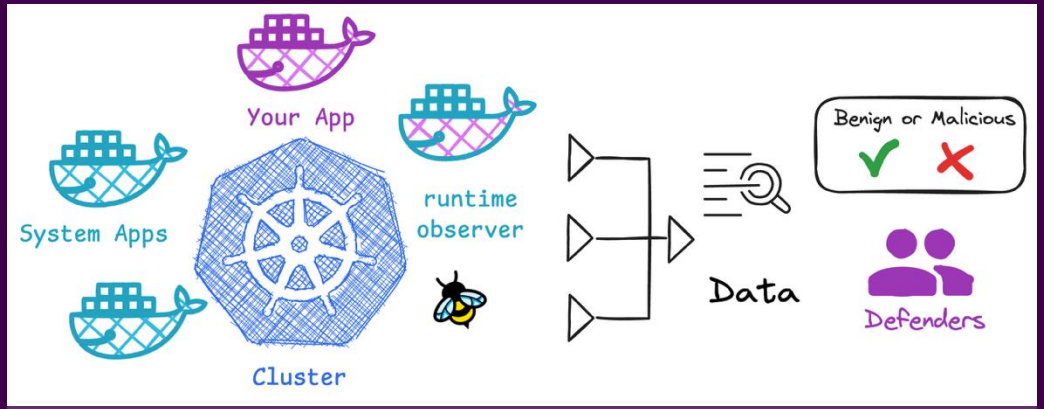
How do you detect tampering?

observe: get some data out

analyze: was this stuff bad?

You use RULES !

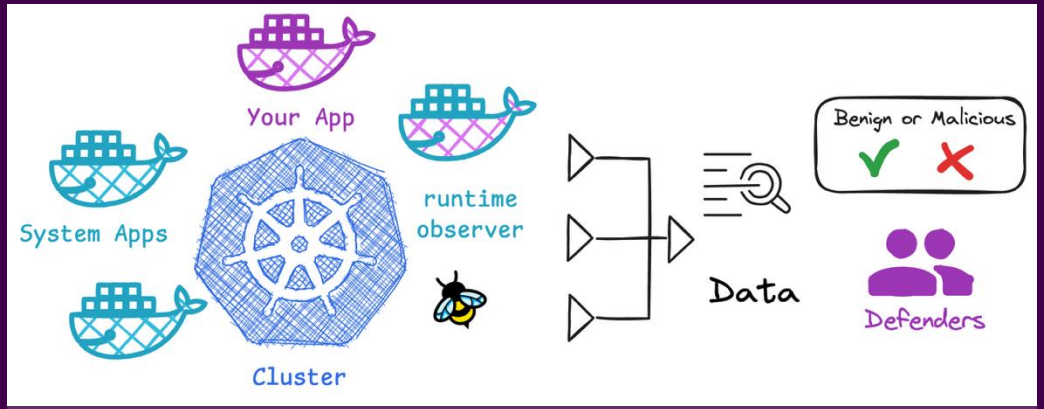
k8sstormcenter



Who writes these rules?

Currently: you the user

k8sstormcenter



Who should write these rules?

Receive and modify the rules: you the user

Create and maintain the rules: the vendor or supplier

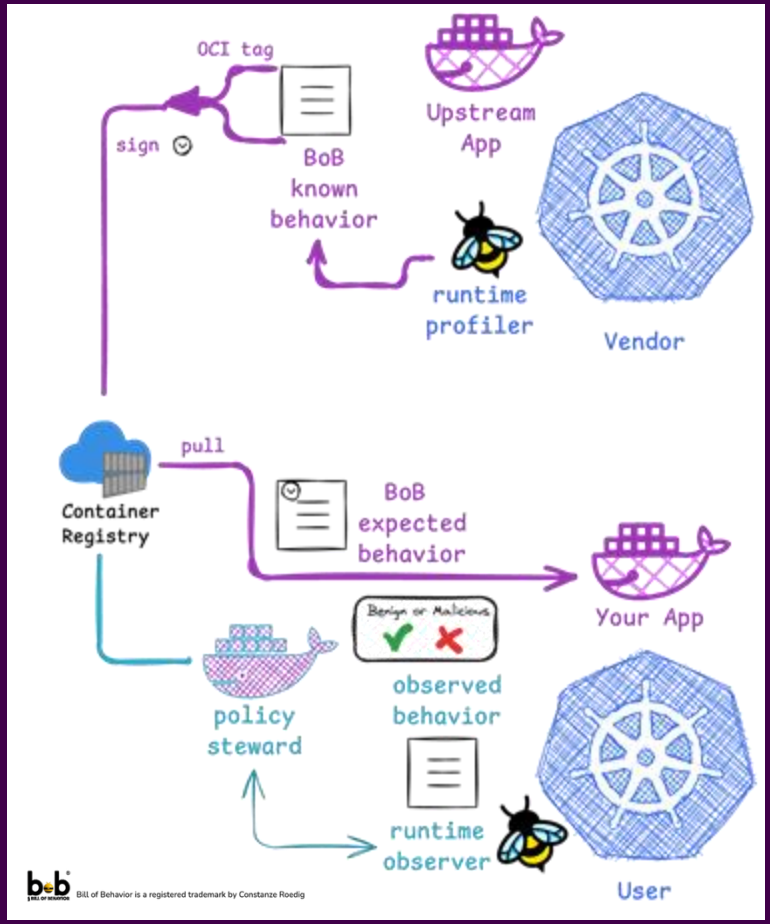
The people who made the SW

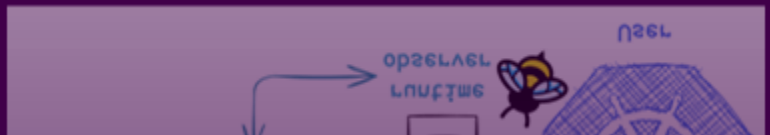
k8sstormcenter

Transferability??!

it works on my cluster

BUT: will it work on your cluster



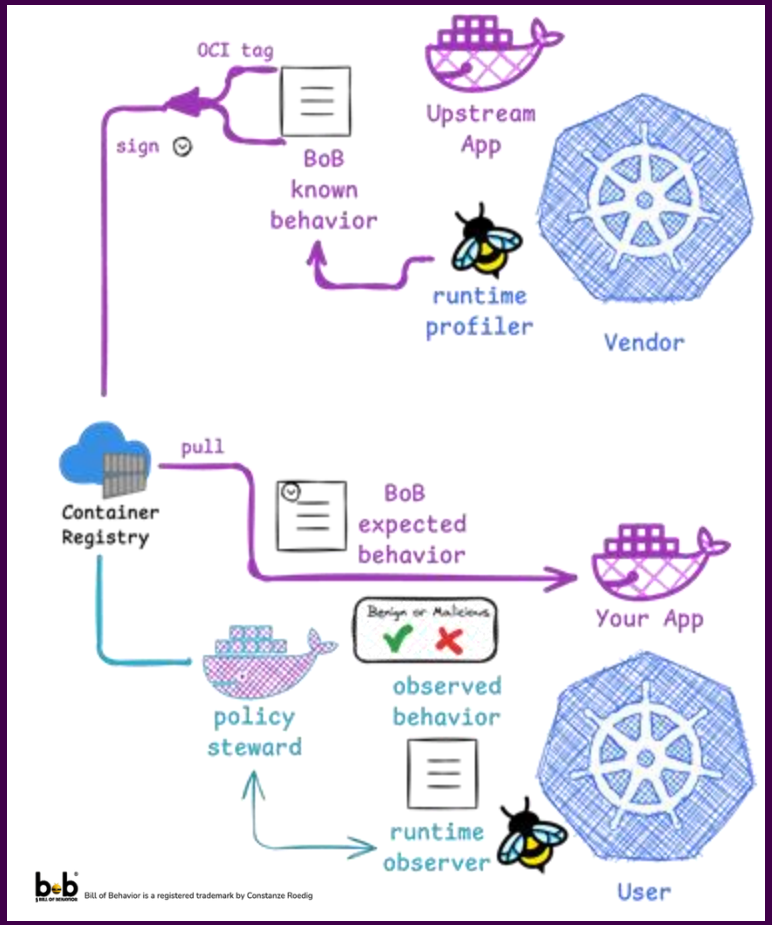


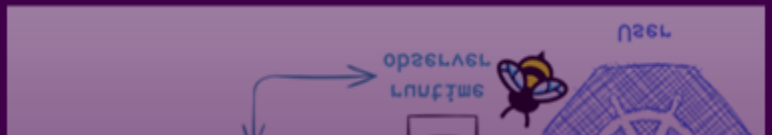
k8sstormcenter

Idea of the year

attach a Bill of Behavior to software

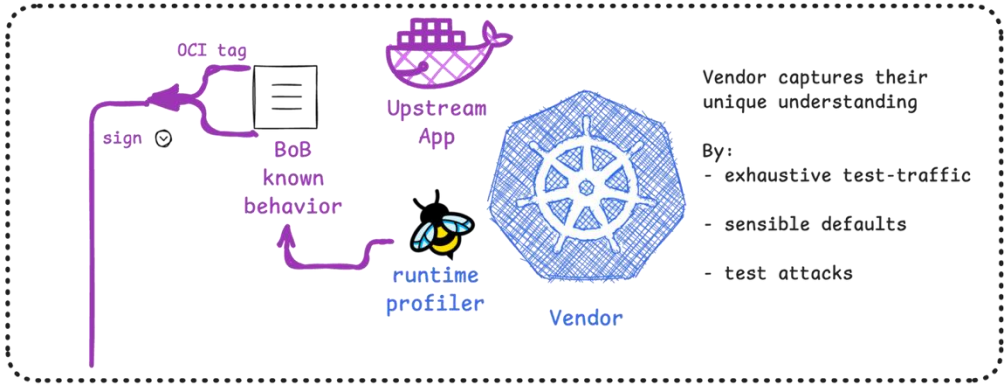
to “supply” benign runtime rules

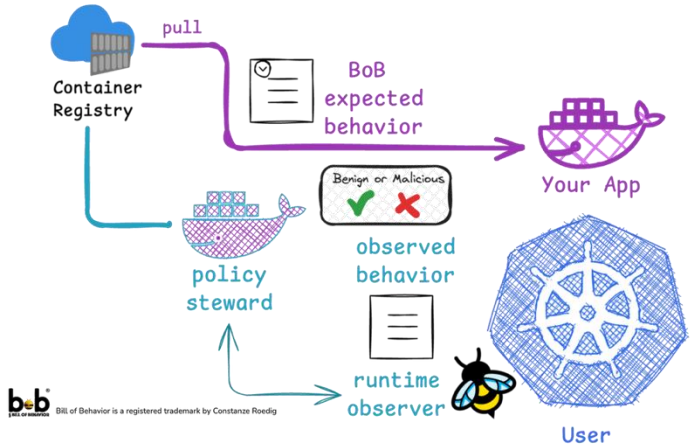




k8sstormcenter

Vendor produces a BoB
in spdx spec!

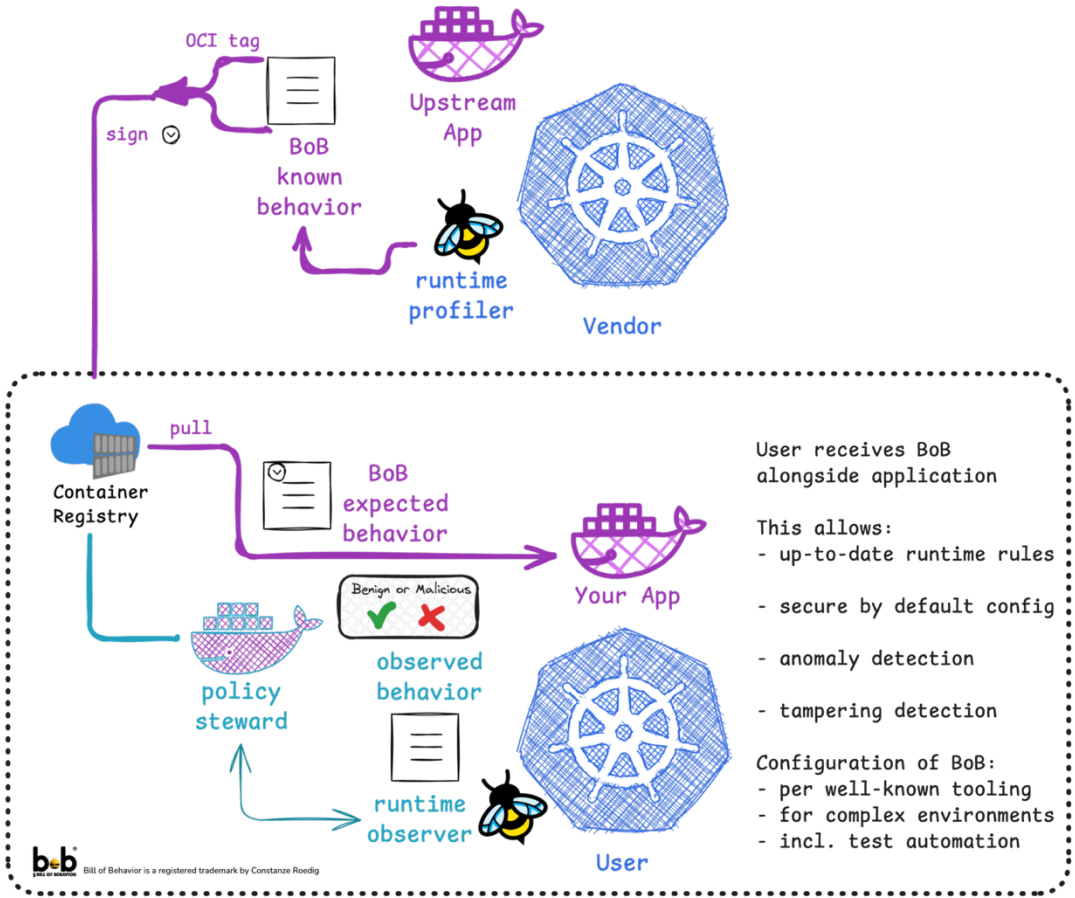




README Apache-2.0 license

```
apiVersion: spdx.softwarecomposition.kubescape.io/v1beta1
kind: ApplicationProfile
metadata:
  name: bob-application123
  namespace: {{ .Release.Namespace }}
spec:
  architectures:
  - amd64
  containers:
  - capabilities:  # KNOWN CAPABILITIES
    - DAC_OVERRIDE
    - SETGID
    - SETUID
    endpoints:     # KNOWN NETWORK
    - direction: inbound
      endpoint: :8080/ping.php
      headers:
        Host: # User accessible Overrides
        - {{ include "mywebapp.fullname" . }}.{{ .Release.Namespace }}.svc.cluster.local:{{ .V
      internal: false
      methods:
      - GET
    execs:       #KNOWN EXEC
    - args:
      - /usr/bin/dirname
      - /var/lock/apache2
      path: /usr/bin/dirname
    - args:
      - /bin/sh
      - -c
      - ping -c 4 172.16.0.2
      path: /bin/sh
    imageID: ghcr.io/k8sstormcenter/webapp@sha256:e323014ec9befb76bc551f8cc3bf158120150e2e277b
    opens:     #KNOWN FILE OPENS
    - flags:
      - O_CLOEXEC
      - O_DIRECTORY
      - O_NONBLOCK
      - O_RDONLY
      path: /etc/apache2/* #globs that generalize UUIDs or well-known FS structures
    - flags:
      - O_CLOEXEC
      - O_RDONLY
      path: /etc/group
    - flags:
      - O_CLOEXEC
      - O_RDONLY
      path: /etc/ld.so.cache
    rulePolicies:  # SPECIFIC EXCEPTION RULES
      R0001:
        processAllowed:
        - ping
        - sh
      R0002: {}
      ...
    syscalls:    # KNOWN SYSCALLS
    - accept4
    - access
    - arch_prctl
    - getegid
```

User receives the BoB with each update



How much does it reduce the attack surface?

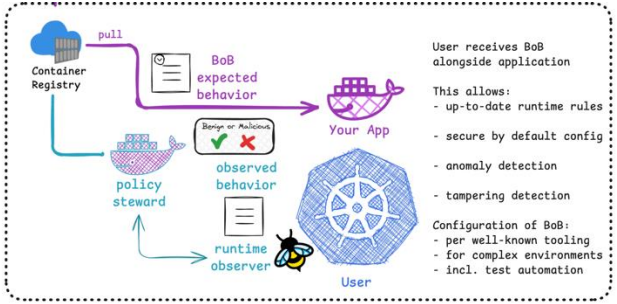
Or should we say: hiding surface?

```
❯ ./attacksurface.sh
```

Profile	Capabilities	Network	Opens (#)	Execs (#)	Allowed Syscalls
Kubernetes Default (v1.33)	unconfined	CNI	unconfined	unconfined	363
catalogsource	CHOWN,DAC_OVERRIDE,DAC_READ_SEARCH,NET_ADMIN,SETGID,SETPCAP,SETUID,SYS_ADMIN	0	19	2	106
kelvin	DAC_OVERRIDE,DAC_READ_SEARCH,NET_ADMIN	0	97	1	76
pem	BPF,DAC_READ_SEARCH,IPC_LOCK,NET_ADMIN,PERFMON,SYS_ADMIN,SYS_PTRACE,SYSLOG	0	390	1	122
pletcd	NET_ADMIN,NET_RAW,SETGID,SETPCAP,SETUID,SYS_ADMIN	0	39	10	92
plnats	DAC_OVERRIDE,DAC_READ_SEARCH,NET_ADMIN	3	11	1	57
querybroker	DAC_OVERRIDE,DAC_READ_SEARCH,FOWNER,NET_ADMIN	0	20	1	107
viziercloudconnector	DAC_OVERRIDE,DAC_READ_SEARCH,NET_ADMIN	0	18	1	70
viziermeta	NET_ADMIN	0	16	1	65
vizieroperator	NET_ADMIN	1	13	1	104

CNCF Pixie provides real-time observability and debugging (based on eBPF)
pem = Pixie edge module (the agent that handles the tracing)

Enforcing vs Alerting




```
❯ ./attacksurface.sh
Profile                 Capabilities                                                    Network  Opens (#)  Execs (#)  Allowed Syscalls
-------                 ------------                                                    -------  ---------  ---------  ----------------
pem                     BPF,DAC_READ_SEARCH,IPC_LOCK,NET_ADMIN,PERFMON,SYS_ADMIN,SYS_PTRACE,SYSLOG  0        390        1          122
-----
```

Anomaly Alerting for:

- Network Endpoints, RunTimeRules
- File Opens and SysCalls

RunTime Engines:

- Kubescape 4.0 (Q4 2025)

Enforcing for:

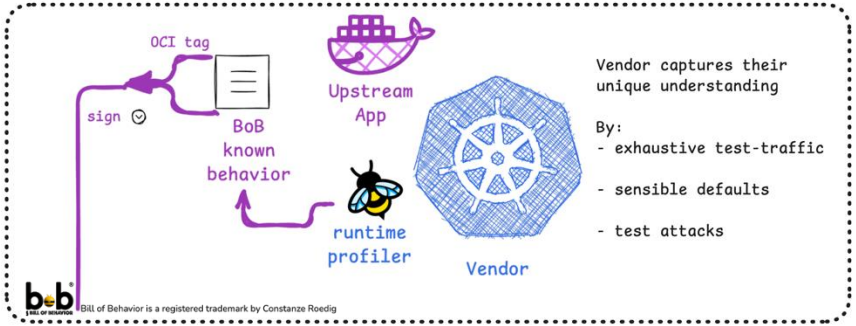
- {Capabilities, Executables} tuples

RunTime Engines:

- AppArmor (roadmap)

Vendor added value

Provide/ship positive tests



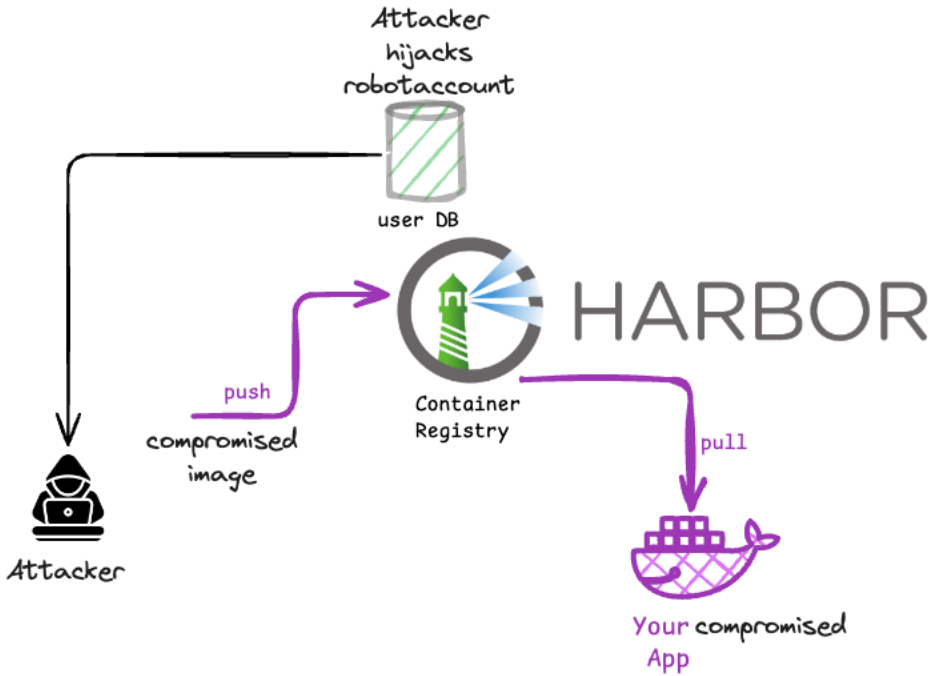

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: "{{ include "redis-bob.fullname" . }}-test"
  labels:
    {{- include "redis-bob.labels" . | nindent 4 }}
    kubescape.io/ignore: "true"
  annotations:
    "helm.sh/hook": test
    "helm.sh/hook-delete-policy": hook-succeeded,hook-failed
spec:
  restartPolicy: Never
  containers:
    - name: redis-test
      image: redis:7.2-alpine
      command:
        - /bin/sh
        - -c
        - |
          set -ex

          REDIS_HOST="{{ .Release.Name }}-redis-master"
          SERVICE="{{ include "redis-bob.fullname" . }}"
          NAMESPACE="{{ .Release.Namespace }}"
          URL="${REDIS_HOST}.${NAMESPACE}.svc.cluster.local:6379"
          REDIS_PASSWORD="vivamusatqueamemus"

          echo "1. PING - Healthcheck"
          redis-cli -h $REDIS_HOST -p $REDIS_PORT  PING | grep PONG

          echo "2. INFO - Get server info"
          redis-cli -h $REDIS_HOST -p $REDIS_PORT  INFO SERVER | grep -q "redis_version"
```

To provide the user with:

- Validation of their alerting / incident response
- Verification of an install (and other traditional use-cases)

Vendor added value

Provide/ship attack tests



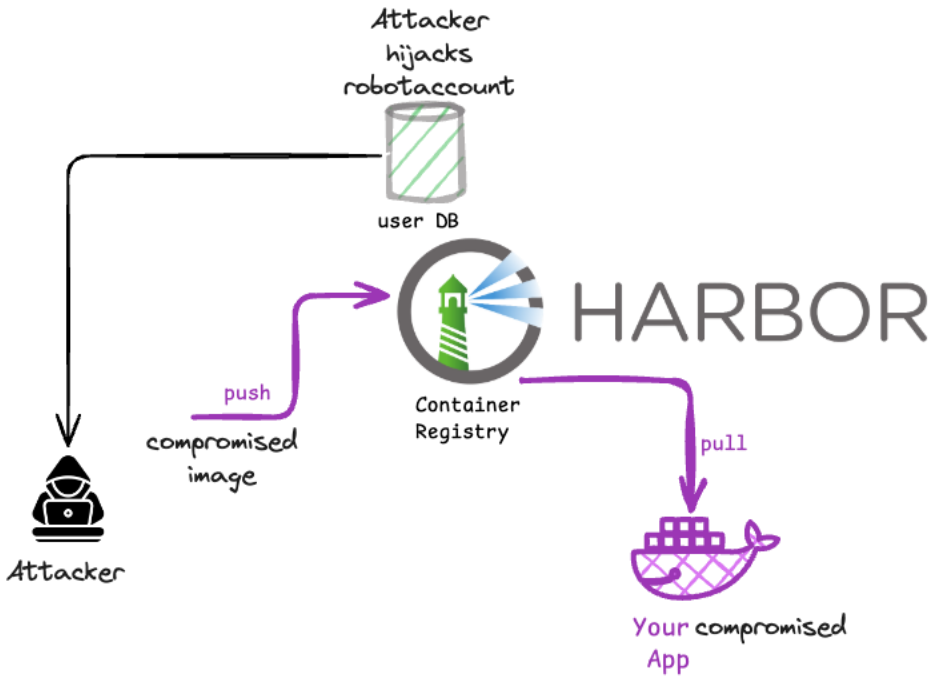
To provide the user with:

- The most critical attack scenarios
- An implementation of testing the threat model detection

User receive maintainers' knowledge of system critical attack paths

Vendor added value

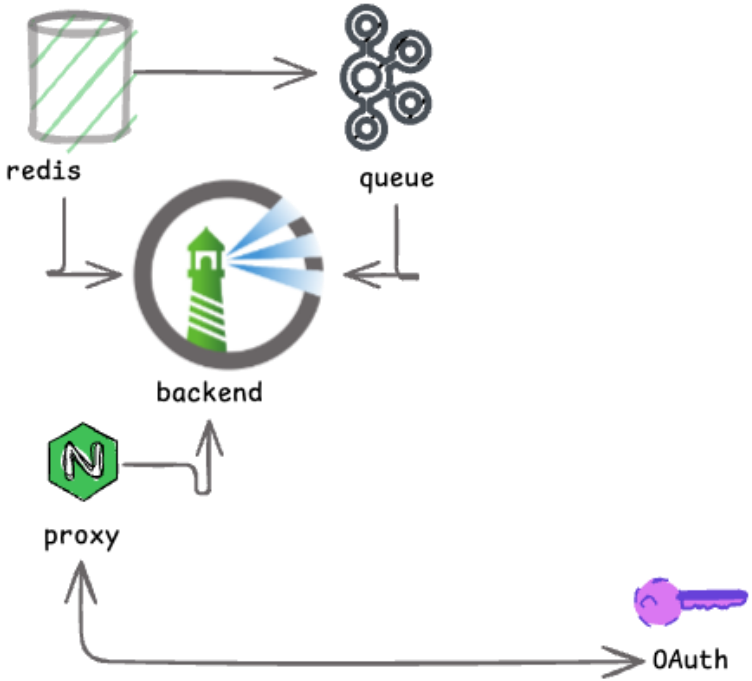
Attack tests package threat model



The Criticality of Harbor Threats

- Exploiting authorization flaws to gain control
- Subversive Malicious Artifact Injection (The Supply Chain Backdoor)
- Vulnerability Scanning Evasion
- Targeted Resource Exhaustion (Operational Denial of Service)

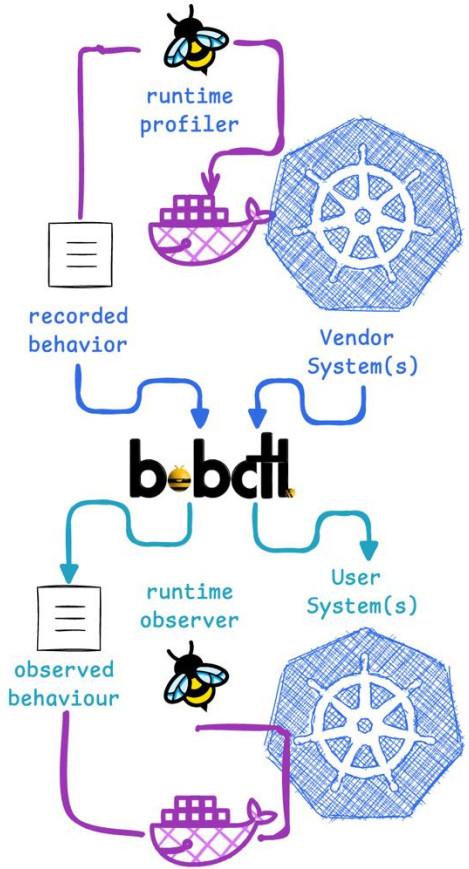
What's the granularity?
Image or helm chart?



Helm Chart ! (or equivalent)

- Config matters
- Privileges matter
- Mount propagations
- Runtime Classes
- Routing choices (gateway, ingress, TLS..)

BoBctl translates between systems



In order to allow for vendor neutrality:

Packaging systems:

- Helm

RunTime Engines:

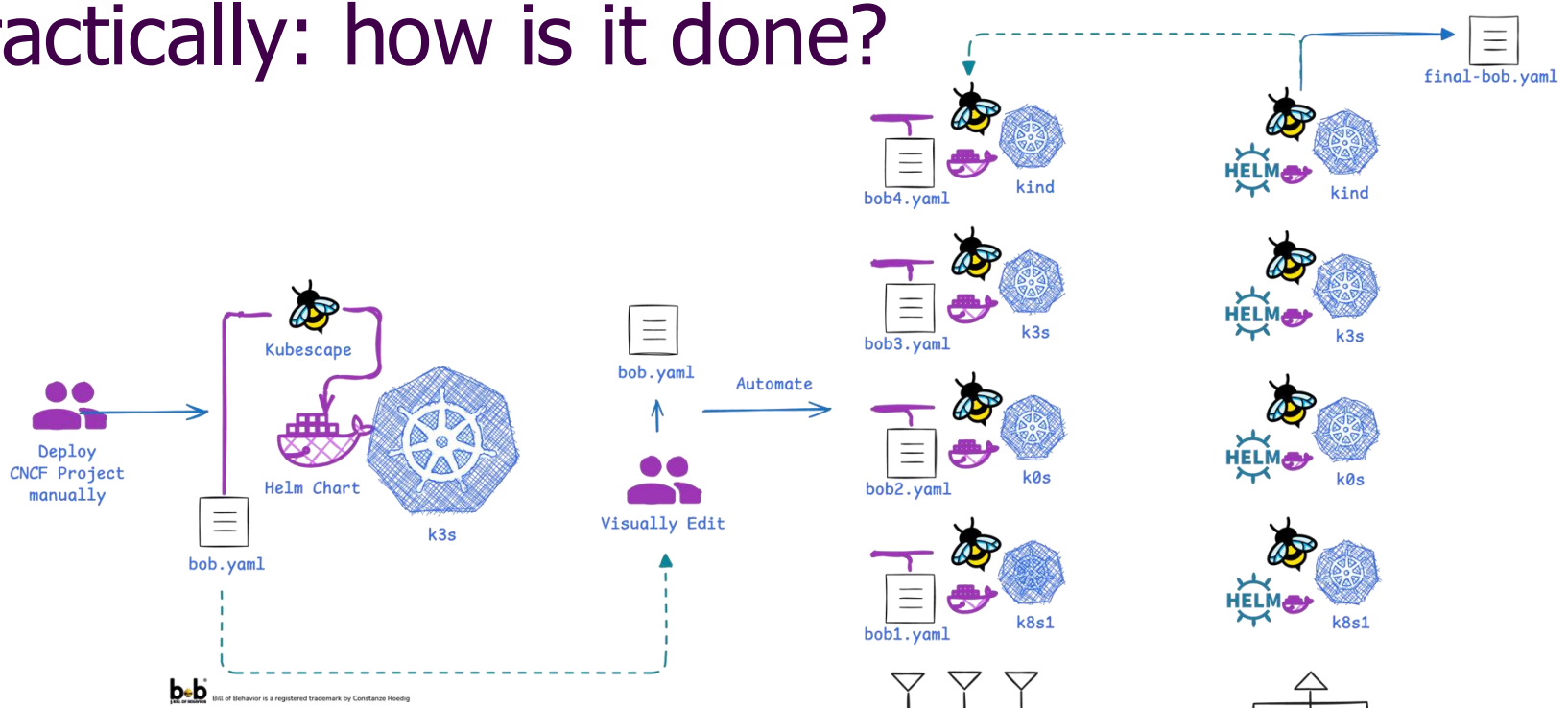
- Kubescape 4.0 (Oct 2025)
- *NeuVector (possible)*
- *AppArmor (planned)*

Kubernetes:

- K8s: GKE, Vanilla, Talos, k3s, k0s
- Kind, Minikube
- *Openshift, SAP … (planned)*

https://github.com/k8sstormcenter/bobctl

Practically: how is it done?



Supersets:

- Detailed for supply chain
- Generalized for runtime

SBoB makes behavior prescriptive

No longer descriptive

Ask your primary care engineer
in case of unexpected side-channels

Tracing has been possible since the dawn of time: but

Instead of everyone recording profiles (potentially incl attacker behavior)

Explicit positives! Makes a DB look like a DB (and a debug pkg like a debug pkg)

Explicit negatives! If negative tests are supplied, users can verify their incident response

some room: to make it transfer well and allow good UX

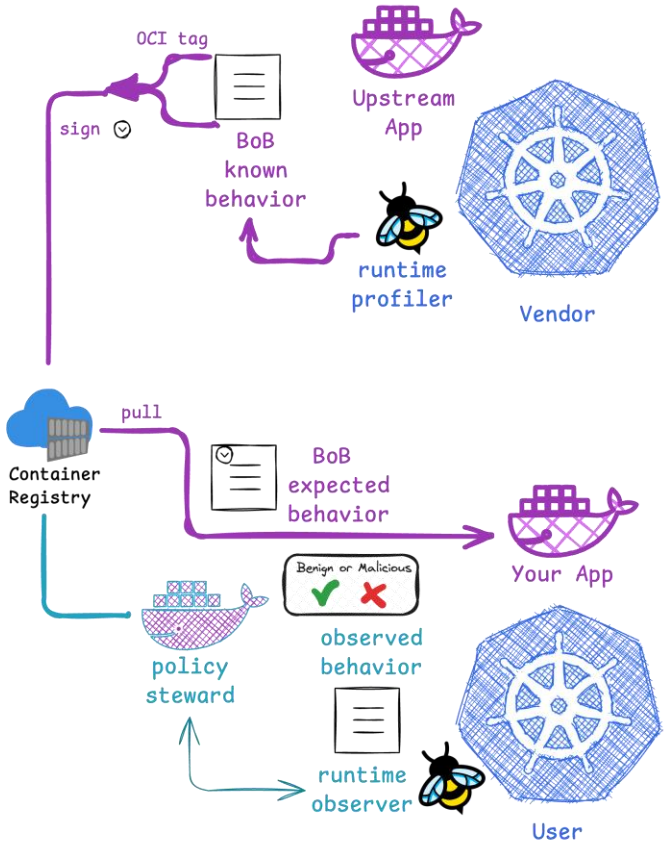
WANTED:
CNCF projects
with their test-suite

needs YOU !

k8sstormcenter

Roadmap

Edge cases, caveats



Research-Collaborations welcome!

Research funding requested for

- Additivity (e.g. various Sidecars like istio) ☑
- Large Parameter Studies ✅ TU WIEN TECHNISCHE UNIVERSITÄT WIEN
- UX Evaluation and Implementation
- Optimal Granularity 👩‍🎓 FH CAMPUS WIEN UNIVERSITY OF APPLIED SCIENCES

  (Performance vs Alert Fatigue)
- Integration into cloudnative ecosystem
- Maintenance

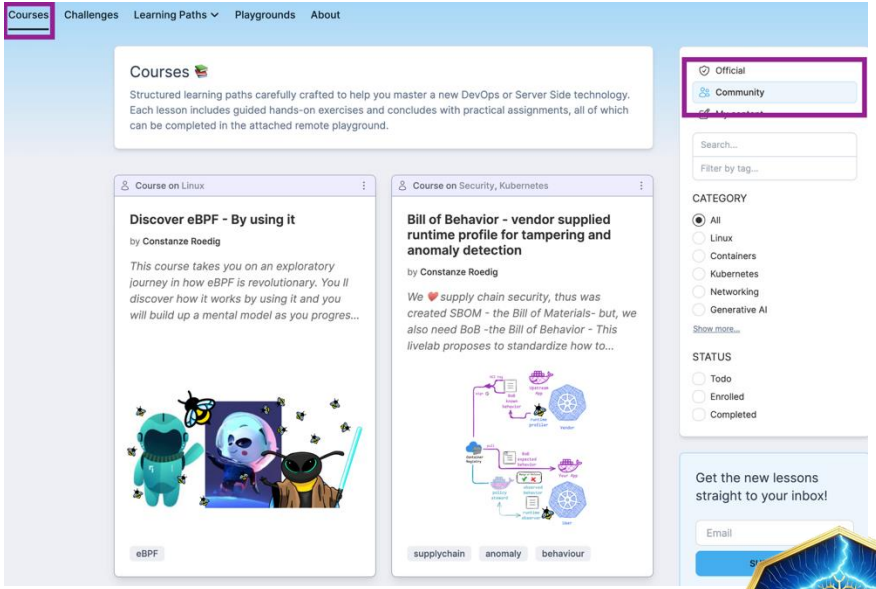
How you can benefit today
Don’t believe it … try out the lab

CNCF SW with test-suite WANTED

Bill of Behavior - vendor supplied runtime profile for tampering and anomaly detection

supplychain anomaly behaviour oci ebpf

We ❤️ supply chain security, thus was created SBOM - the Bill of Materials- but, we also need BoB -the Bill of Behavior - This livelab proposes to standardize how to record a benign behavior profile, extract, sign and publish it, for users to consume, ingest, verify it and detect attacks and tampering. For software-vendors, a BoB creates trust and transparency, For users, it makes anomaly detection realistically achievable.



https://labs.iximiuz.com/courses/bill-of-behaviour-c070da3a

Summary

Ask your primary care engineer
in case of unexpected effects

Container indlægsseddel

BoB: Software Bill of Behavior

What? generalized runtime profile encoding benign syscalls, paths, network, capas, execs

Who? Vendors attach to software at “the factory”

Why? Allows users to inherit maintained, up-to-date rules for anomaly detection

so that: Users stop play catch-up

CLOUD NATIVE DENMARK
October 7-8, 2025 • Aarhus

CNDA
Cloud Native Days Austria

ITSECURITY SUMMIT

Thank you !!

SOFIA | BULGARIA KCD
GROWING CLOUD NATIVE TOGETHER

Redpill Linpro Cloud

iximiuz Labs

SBoB would not have been possible without: Matthias, Ben, Thomas, Norman, Stefan, Peter, Markus, David, Pepijn, Mario, Ivan, Markus, Stephie, Andi, Ernst, Michi, Reini and Alois

https://cloudnativecompass.dev

https://www.letsboot.ch

https://kubescape.io

https://github.com/k8sstormcenter/bobctl

Star it if you want it